

Understanding Exception Handling in Bus Booking System

Bus Booking System — Full Stack Application

Angular .NET Web API · PostgreSQL

Assignment Task

Please visit your project and understand how exceptions are being handled. Please write how you think exception is being handled in the project by AI.

1. Project Overview

Bus Booking System is an end-to-end web-based application that mimics actual bus travels booking processes in the real world. This application has been developed using **Angular (frontend)**, **.NET 10 Web API (backend)**, and **PostgreSQL (database)**.

There are three types of users supported by the application – **Customers, Operators, and Admin**.

1.1 Folder Structure

The backend is cleanly organised into dedicated folders, each with a single responsibility:

Folder	Purpose
Backend/Models/	C# entity classes that map to PostgreSQL tables (Booking, Bus, User, Feedback, ...)
Backend/Controllers/	ASP.NET controllers that expose REST API endpoints
Backend/Services/	Business-logic classes (EmailService) — separated from controllers
Backend/Interfaces/	C# interfaces that define service contracts (IEmailService) ← NEW
Backend/Data/	Entity Framework AppDbContext — database session / DbSet definitions
Frontend/src/app/pages/	Angular page components (Home, Login, Seat, Payment, Ticket, ...)
Frontend/src/app/components/	Shared UI components (Navbar)
Frontend/src/app/services/	Angular services that call the backend API

2. OOP Concepts Learned Today

1. Project Overview

Bus Booking System is an end-to-end web-based application that mimics actual bus travels booking processes in the real world. This application has been developed using **Angular (frontend)**, **.NET 10 Web API (backend)**, and **PostgreSQL (database)**.

There are three types of users supported by the application – **Customers, Operators, and Admin**.

1.1 Folder Structure

The backend is cleanly organised into dedicated folders, each with a single responsibility:

Folder	Purpose
Backend/Models/	C# entity classes that map to PostgreSQL tables (Booking, Bus, User, Feedback, ...)
Backend/Controllers/	ASP.NET controllers that expose REST API endpoints
Backend/Services/	Business-logic classes (EmailService) — separated from controllers
Backend/Interfaces/	C# interfaces that define service contracts (IEmailService) ← NEW
Backend/Data/	Entity Framework ApplicationDbContext — database session / DbSet definitions
Frontend/src/app/pages/	Angular page components (Home, Login, Seat, Payment, Ticket, ...)
Frontend/src/app/components/	Shared UI components (Navbar)
Frontend/src/app/services/	Angular services that call the backend API

This document explains how structured exception handling, custom exceptions, try-catch-finally blocks, and middleware-based error handling are implemented in the Bus Booking System project.

2. OOP Concepts Learned Today

The below listed fundamental concepts of OOP were discussed in today's lecture:

1. **Exception Handling** – Handling unexpected situations during runtime error. There are two kinds of exception handling -> Structured and Unstructured – try-catch-finally block.
2. **Separation of Concerns**

1. Exception Handling Before Improvements

Initially, the project had only basic try-catch usage in a few places such as email sending. Most database operations and API actions had no structured exception handling. This caused raw HTTP 500 errors and unformatted stack traces to be returned to the Angular frontend.

- Old email handling pattern:

```
try
{
    emailService.SendBookingEmail(user.Email, subject, body);
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

2. Custom Exceptions

Custom exceptions were introduced to represent business-specific failure scenarios. Instead of throwing generic exceptions, meaningful typed exceptions are now used.

Exception	Purpose	HTTP Status
SeatAlreadyTakenException	Seat already booked or locked	409
SeatLockExpiredException	5-minute lock expired	410
BookingNotFoundException	Booking/group not found	404
EmailDeliveryException	SMTP email sending failed	502

- Example usage in BookingController.cs:

```
if (exists)
    throw new SeatAlreadyTakenException(booking.BusId, booking.SeatNumber);
```

3. Try-Catch-Finally Blocks

Structured exception handling was added to EmailService.cs. Specific exceptions are caught separately and the finally block guarantees SMTP cleanup.

- Structured exception handling in EmailService.cs:

```
try
{
    smtp.Connect("smtp.gmail.com", 587, false);
    smtp.Authenticate(_fromEmail, _appPassword);
}
```

```
smtp.Send(message);  
}  
catch (FormatException ex)  
{  
    throw new EmailDeliveryException(toEmail, ex);  
}  
catch (AuthenticationException ex)  
{  
    throw new EmailDeliveryException(toEmail, ex);  
}  
finally  
{  
    if (smtp.IsConnected)  
        smtp.Disconnect(true);  
  
    smtp.Dispose();  
}
```

4. Global Exception Middleware

A middleware layer was introduced to centralize exception handling for the entire application.

This prevents raw stack traces from being exposed to the frontend and ensures consistent JSON error responses.

- Middleware registration in Program.cs:

```
app.UseMiddleware<GlobalExceptionMiddleware>();
```

- Global catch structure:

```
catch (BusBookingException ex)  
{  
    await WriteErrorResponse(context, ex.StatusCode, ex.Message);  
}  
catch (DbUpdateException ex)  
{  
    await WriteErrorResponse(context, 500, "Database error occurred.");  
}  
catch (Exception ex)  
{  
    await WriteErrorResponse(context, 500, "Unexpected error occurred.");  
}
```

5. Folder Structure

The project structure was improved to separate exception-related concerns cleanly.

- Updated backend structure:

```
Backend/  
├── Controllers/  
│   ├── BookingController.cs  
│   ├── UserController.cs  
│   └── FeedbackController.cs  
├── Exceptions/  
│   ├── BusBookingException.cs  
│   ├── SeatAlreadyTakenException.cs  
│   ├── SeatLockExpiredException.cs  
│   ├── BookingNotFoundException.cs  
│   └── EmailDeliveryException.cs  
├── Middleware/  
│   └── GlobalExceptionHandler.cs  
├── Services/  
│   └── EmailService.cs  
└── Program.cs
```

6. How Exception Handling Helps the Project

The new structured exception handling system significantly improves reliability, debugging, and user experience.

Problem Before	Improvement After
Raw 500 errors	Middleware returns clean JSON responses
SMTP connection leaks	finally block guarantees disconnect
Generic catch(Exception)	Specific exception types used
Silent email failures	EmailDeliveryException introduced
No seat conflict handling	SeatAlreadyTakenException added

7. Final Understanding

The project now treats exceptions as structured business events rather than plain error messages.

Custom exceptions improve readability, middleware centralizes handling, and try-catch-finally ensures system stability.

The most important learning was understanding that exception handling is not just about preventing crashes —

Name – Sai Nikitha NSR
Day – 10 Assignment

Date – 6th May, 2026
Mail – sainikinsr@gmail.com

it is about making the system predictable, maintainable, and user-friendly.

OOPS in Project · Bus Booking System · Angular + .NET + PostgreSQL